

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Институт информатики и вычислительной техники

09.03.01 "Информатика и вычислительная техника"
профиль "Программное обеспечение средств
вычислительной техники и автоматизированных систем"

Кафедра прикладной математики и кибернетики

РГР по дисциплине
Алгоритмы и вычислительные методы оптимизации

Вариант 3

Выполнил:

студент гр.ИП-312

_____/Дорогин Н. С./
ФИО студента

«6» мая 2026 г.

Новосибирск 2026 г.

- программа работает с классом простых дробей;
- программа находит решение ЗЛП методом по варианту;
- в качестве входных данных подаются матрицы, удовлетворяющие требованиям метода;
- программа должна обрабатывать возможное отсутствие решений;
- если решений бесконечно много, то программа должна записать общий вид этих решений;

Результаты работы программы

```
РГР > examples > ≡ nosol.txt
      You, 5 hours ago | 1 author (You)
1  -2 -1 1 0 0 | -4
2  -1 -1 0 1 0 | -3
3   1 2 0 0 1 | -5
4  Z:
5  1 1 0 0 0 | min
```

Скриншот 1. Задача без решения

Введите имя файла с матрицей (или exit, чтобы выйти):
nosol

Исходная задача:

$Z = x_1 + x_2 \rightarrow \min$
| $-2x_1 - x_2 + x_3 = -4$
| $-x_1 - x_2 + x_4 = -3$
| $x_1 + 2x_2 + x_5 = -5$

Преобразованная задача:

$-Z = -x_1 - x_2 \rightarrow \max$
| $-2x_1 - x_2 + x_3 = -4$
| $-x_1 - x_2 + x_4 = -3$
| $x_1 + 2x_2 + x_5 = -5$

Таблица № 1:

Б.п	1	x_1	x_2	x_3	x_4	x_5
x_3	-4	-2	-1	1	0	0
x_4	-3	-1	-1	0	1	0
x_5	-5	1	2	0	0	1
$-Z$	0	1	1	0	0	0
C0	-	-	-	-	-	-

Соответствующее решение:

$Z(0, 0) = 0$

Итоговое решение:

Нет решения!

Скриншот 2. Тест на задаче без решения

```

РГР > examples > ≡ n1.txt
You, 22 hours ago | 1 auth
1  4 7 -1 0 0 | 57
2  4 1 0 -1 0 | 15
3  3 11 0 0 -1 | 60
4  Z:
5  1 7 0 0 0 | min
6

```

Скриншот 3. Задача с единственным решением

```

Исходная задача:
Z = x1 + 7*x2 -> min
| 4*x1 + 7*x2 - x3 = 57
| 4*x1 + x2 - x4 = 15
| 3*x1 + 11*x2 - x5 = 60

Преобразованная задача:
-Z = -x1 - 7*x2 -> max
| -4*x1 - 7*x2 + x3 = -57
| -4*x1 - x2 + x4 = -15
| -3*x1 - 11*x2 + x5 = -60

Таблица № 1:
+-----+-----+-----+-----+-----+-----+-----+
| Б.п   |      1      |      x1      |      x2      |      x3      |      x4      |      x5      |
+-----+-----+-----+-----+-----+-----+-----+
| x3     |      -57     |      -4      |      -7      |      1       |      0       |      0       |
+-----+-----+-----+-----+-----+-----+-----+
| x4     |      -15     |      -4      |      -1      |      0       |      1       |      0       |
+-----+-----+-----+-----+-----+-----+-----+
| x5     |      -60     |      [-3]    |      -11     |      0       |      0       |      1       |
+-----+-----+-----+-----+-----+-----+-----+
| -Z     |      0       |      1       |      7       |      0       |      0       |      0       |
+-----+-----+-----+-----+-----+-----+-----+
| CO     |      -       |      (1/3)   |      (7/11)  |      -       |      -       |      -       |
+-----+-----+-----+-----+-----+-----+-----+

Соответствующее решение:
Z(0, 0) = 0

```

Скриншот 4. Тест на задаче с единственным решением (Часть 1)

Соответствующее решение:

$$Z(0, 0) = 0$$

Разрешающий элемент: -3

Выводим из базиса x_5

Вводим в базис x_1

Таблица № 2:

Б.п	1	x_1	x_2	x_3	x_4	x_5
x_3	23	0	$(23/3)$	1	0	$(-4/3)$
x_4	65	0	$(41/3)$	0	1	$(-4/3)$
x_1	20	1	$(11/3)$	0	0	$(-1/3)$
-Z	-20	0	$(10/3)$	0	0	$(1/3)$
CO	-	-	-	-	-	-

Соответствующее решение:

$$Z(20, 0) = 20$$

Итоговое решение:

$$Z_{\min} = Z(20, 0) = 20$$

Введите имя файла с матрицей (или exit, чтобы выйти):

Скриншот 5. Тест на задаче с единственным решением (Часть 2)

inf

Исходная задача:

$$Z = 2x_1 + 4x_2 \rightarrow \min$$

$$x_1 + 2x_2 - x_3 = 4$$

$$2x_1 + 4x_2 - x_4 = 8$$

$$x_1 + x_2 + x_5 = 3$$

Преобразованная задача:

$$-Z = -2x_1 - 4x_2 \rightarrow \max$$

$$-x_1 - 2x_2 + x_3 = -4$$

$$-2x_1 - 4x_2 + x_4 = -8$$

$$x_1 + x_2 + x_5 = 3$$

Таблица № 1:

Б.п	1	x_1	x_2	x_3	x_4	x_5
x_3	-4	-1	-2	1	0	0
x_4	-8	[-2]	-4	0	1	0
x_5	3	1	1	0	0	1
-Z	0	2	4	0	0	0
CO	-	1	1	-	-	-

Соответствующее решение:

$$Z(0, 0) = 0$$

Скриншот 6. Тест на задаче с бесконечным множеством решений (Часть 1)

Соответствующее решение:

$$Z(0, 0) = 0$$

Разрешающий элемент: -2

Выводим из базиса x_4

Вводим в базис x_1

Таблица № 2:

Б.п	1	x_1	x_2	x_3	x_4	x_5
x_3	0	0	0	1	$(-1/2)$	0
x_1	4	1	2	0	$(-1/2)$	0
x_5	-1	0	$[-1]$	0	$(1/2)$	1
-Z	-8	0	0	0	1	0
CO	-	-	0	-	-	-

Соответствующее решение:

$$Z(4, 0) = 8$$

Скриншот 7. Тест на задаче с бесконечным множеством решений (Часть 2)

Разрешающий элемент: -1

Выводим из базиса x_5

Вводим в базис x_2

Таблица № 3:

Б.п	1	x_1	x_2	x_3	x_4	x_5
x_3	0	0	0	1	$(-1/2)$	0
x_1	2	1	0	0	$(1/2)$	2
x_2	1	0	1	0	$(-1/2)$	-1
-Z	-8	0	0	0	1	0
CO	-	-	-	-	-	-

Соответствующее решение:

$$Z(2, 1) = 8$$

Итоговое решение:

Решений бесконечно много:

$$Z_{\min} = Z(2 + 2 \cdot \lambda, 1 - \lambda) = 8$$

где $\lambda \in [0, 1]$ - параметр

Введите имя файла с матрицей (или exit, чтобы выйти):

exit

Завершение работы...

PS C:\Users\Danik\Desktop\Учёба\ABMO\sibsutis_09.03.01_IP_3_course_AVM0\PGP> |

Скриншот 8. Тест на задаче с бесконечным множеством решений (Часть 3).

Выход из программы

Исходный код программы

Fract.py

```
import math

class Fract:
    def __init__(self, upper, lower=1):
        if lower == 0:
            raise ValueError("Знаменатель не может быть 0!")
        if lower < 0:
            upper = -upper
            lower = -lower
        self.upper = upper
        self.lower = lower
        self.reduce()

    def __str__(self):
        if self.upper % self.lower == 0:
            return str(self.upper // self.lower)
        else:
            return f'({self.upper}/{self.lower})'

    def reduce(self):
        gcd = math.gcd(self.upper, self.lower)
        self.upper //= gcd
        self.lower //= gcd

    def __mul__(self, x):
        if isinstance(x, Fract):
            return Fract(self.upper * x.upper, self.lower *
x.lower)
        elif isinstance(x, (int, float)):
            return Fract(self.upper * x, self.lower)
        else:
            raise TypeError("Ошибка умножения дроби!")

    def __truediv__(self, x):
        if isinstance(x, Fract):
            return Fract(self.upper * x.lower, self.lower *
x.upper)
        elif isinstance(x, (int, float)):
            return Fract(self.upper, self.lower * x)
        else:
            raise TypeError("Ошибка деления дроби!")
```

```

def __add__(self, x):
    if isinstance(x, Fract):
        new_upper = self.upper * x.lower + x.upper *
self.lower
        new_lower = self.lower * x.lower
        return Fract(new_upper, new_lower)
    elif isinstance(x, (int, float)):
        return Fract(self.upper + x * self.lower,
self.lower)
    else:
        raise TypeError("Ошибка сложения дробей!")

def __sub__(self, x):
    if isinstance(x, Fract):
        new_upper = self.upper * x.lower - x.upper *
self.lower
        new_lower = self.lower * x.lower
        return Fract(new_upper, new_lower)
    elif isinstance(x, (int, float)):
        return Fract(self.upper - x * self.lower,
self.lower)
    else:
        raise TypeError("Ошибка вычитания из дроби!")

def __eq__(self, x):
    if isinstance(x, Fract):
        return self.upper * x.lower == x.upper *
self.lower
    elif isinstance(x, (int, float)):
        return self.upper == x * self.lower
    return False

def __ne__(self, x):
    return not self.__eq__(x)

def __gt__(self, x):
    if isinstance(x, Fract):
        return self.upper * x.lower > x.upper * self.lower
    elif isinstance(x, (int, float)):
        return self.upper > x * self.lower
    raise TypeError("Ошибка сравнения!")

def __ge__(self, x):
    if isinstance(x, Fract):
        return self.upper * x.lower >= x.upper *
self.lower

```



```

elif isinstance(x, (int, float)):
    return self.upper >= x * self.lower
raise TypeError("Ошибка сравнения!")

def __lt__(self, x):
    if isinstance(x, Fract):
        return self.upper * x.lower < x.upper * self.lower
    elif isinstance(x, (int, float)):
        return self.upper < x * self.lower
    raise TypeError("Ошибка сравнения!")

def __le__(self, x):
    if isinstance(x, Fract):
        return self.upper * x.lower <= x.upper *
self.lower
    elif isinstance(x, (int, float)):
        return self.upper <= x * self.lower
    raise TypeError("Ошибка сравнения!")

def __abs__(self):
    return Fract(abs(self.upper), self.lower)

def __radd__(self, x):
    return self.__add__(x)

def __rsub__(self, x):
    if isinstance(x, (int, float)):
        return Fract(x * self.lower - self.upper,
self.lower)
    else:
        raise TypeError("Ошибка вычитания из дроби!")

def __rmul__(self, x):
    return self.__mul__(x)

def __rtruediv__(self, x):
    if isinstance(x, (int, float)):
        return Fract(x * self.lower, self.upper)
    else:
        raise TypeError("Ошибка деления на дробь!")

```

```
from Fract import Fract

def parse_matrix(matrix_str):
    rows = matrix_str.strip().split('\n')
    finish = False
    matrix = []
    b_vector = []
    z_vector = []
    target = 0

    for row in rows:
        row = row.strip()
        if not row:
            continue

        if row == 'Z:':
            finish = True
            continue

        if not finish:
            if '|' not in row:
                continue

            s_matrix, right = row.split('|')
            left_parts = s_matrix.strip().split()
            left_part = []
            for num in left_parts:
                if '/' in num:
                    up, low = map(int, num.split('/'))
                    left_part.append(Fract(up, low))
                else:
                    left_part.append(Fract(int(num)))

            right_str = right.strip()
            if '/' in right_str:
                up, low = map(int, right_str.split('/'))
                right_part = Fract(up, low)
            else:
                right_part = Fract(int(right_str))

            matrix.append(left_part)
            b_vector.append(right_part)
        else:
            if '|' not in row:
```

continue

```
zs, starget = row.split('|')
if "max" in starget:
    target = 1
elif "min" in starget:
    target = -1

zs_parts = zs.strip().split()
for num in zs_parts:
    if '/' in num:
        up, low = map(int, num.split('/'))
        z_vector.append(Fract(up, low))
    else:
        z_vector.append(Fract(int(num)))

return matrix, b_vector, z_vector, target
```

AmbSimplex.py

```
from Fract import Fract
from MatrixFunctions import parse_matrix

def print_problem(matrix, b_vector, z_vector, target, alt):
    rows = len(matrix)
    cols = len(matrix[0])

    # Вывод целевой функции
    if alt:
        print("-Z = ", end="")
        target *= -1
    else:
        print("Z = ", end="")
    first = True
    for j in range(cols):
        if z_vector[j] != 0:
            k = z_vector[j]
            if not first and k > 0:
                print(" + ", end="")
            elif not first and k < 0:
                print(" - ", end="")
            elif first and k < 0:
                print("-", end="")
            else:
                print(f"x{j+1}", end="")

            abs_k = abs(k)
            if abs_k == 1:
                print(f"x{j+1}", end="")
            else:
                print(f"{abs_k}*x{j+1}", end="")
            first = False

    if target > 0:
        print(" -> max")
    else:
        print(" -> min")

    # Вывод ограничений
    for i in range(rows):
        print("| ", end="")
        first = True
        for j in range(cols):
            if matrix[i][j] != 0:
                k = matrix[i][j]
                if not first and k > 0:
                    print(" + ", end="")
                elif not first and k < 0:
                    print(" - ", end="")
                elif first and k < 0:
                    print("-", end="")
                else:
                    print(f"x{j+1}", end="")

            abs_k = abs(k)
```

```

        if abs_k == 1:
            print(f"x{j+1}", end="")
        else:
            print(f"{abs_k}*x{j+1}", end="")
        first = False

    print(f" = {b_vector[i]}")

def isBasic(x, y, matrix, z_vector):
    for i in range(len(matrix)):
        if ((i != x) and (matrix[i][y]) != 0):
            return False
    if z_vector[y] != 0:
        return False
    return True

def mulRowVal(row, val):
    for i in range(len(row)):
        row[i] *= val

def printTable(matrix, basis, b_vector, z_vector, z, co_vector, rr, rc,
target):
    col_width = 12
    def center(text, width=col_width):
        text = str(text)
        if len(text) > width:
            return text[:width-3] + "..."
        return text.center(width)

    border = "+" + "-" * col_width + "+" + "-" * col_width + "+"

    for _ in range(len(matrix[0])):
        border += "-" * col_width + "+"
    print(border.replace("-", "="))

    header = "|" + center("Б.п") + "|" + center("1") + "|"
    for i in range(len(matrix[0])):
        header += center(f"x{i+1}") + "|"
    print(header)
    print(border.replace("-", "="))

    for x in range(len(basis)):
        row = "|" + center(f"x{basis[x]+1}") + "|" +
center(str(b_vector[x])) + "|"
        for y in range(len(matrix[x])):
            if x == rr and y == rc:
                row += center(f'[{matrix[x][y]}]') + "|"
            else:
                row += center(str(matrix[x][y])) + "|"
        print(row)
        print(border)

```

```

z_row = "|"
if target > 0:
    z_row += center("Z")
else:
    z_row += center("-Z")
z_row += "|" + center(str(z)) + "|"

for y in range(len(matrix[0])):
    z_row += center(str(z_vector[y])) + "|"
print(z_row)
print(border)

co_row = "|" + center("CO") + "|" + center("-") + "|"
if(co_vector):
    for y in range(len(matrix[0])):
        co_row += center(str(co_vector[y])) + "|"
else:
    for y in range(len(matrix[0])):
        co_row += center("-") + "|"
print(co_row)
print(border)

def noNegative(vector):
    for value in vector:
        if value < 0: return False
    return True

def find_res_row(b_vector):
    min = 1
    res = 0
    for i in range(len(b_vector)):
        if b_vector[i] < min:
            min = b_vector[i]
            res = i
    return res

def compute_co(row, basis, z_vector):
    co = []
    for i in range(len(z_vector)):
        if i in basis or row[i] >= 0:
            co.append("-")
        else:
            co.append(abs(z_vector[i]/row[i]))
    return co

def find_res_col(co_vector):
    min = None
    res = -1
    for i in range(len(co_vector)):
        if not isinstance(co_vector[i], str):
            min = co_vector[i]

```

```

        res = i
        break
    if res < 0:
        return res
    for i in range(len(co_vector)):
        if co_vector[i] != "-" and co_vector[i] < min:
            min = co_vector[i]
            res = i
    return res

def new_table(old_matrix, old_b_vector, old_z_vector, old_z_answ, rr, rc,
old_basis):
    rows = len(old_matrix)
    cols = len(old_matrix[0])

    matrix = [[Fract(0) for _ in range(cols)] for _ in range(rows)]
    b_vector = [Fract(0) for _ in range(rows)]
    z_vector = [Fract(0) for _ in range(cols)]
    basis = old_basis[:]
    basis[rr] = rc

    # Делим разрешающую строку на разрешающий элемент
    for col in range(cols):
        matrix[rr][col] = old_matrix[rr][col] / old_matrix[rr][rc]

    # Метод прямоугольников (матрица)
    for x in range(rows):
        for y in range(cols):
            if x == rr or y == rc: continue
            matrix[x][y] = old_matrix[x][y] - ((old_matrix[rr][y] *
old_matrix[x][rc]) / old_matrix[rr][rc])

    # Метод прямоугольников (z-строка)
    for y in range(cols):
        if y != rc:
            z_vector[y] = old_z_vector[y] - ((old_matrix[rr][y] *
old_z_vector[rc]) / old_matrix[rr][rc])

    # Метод прямоугольников (b-строка)
    for x in range(rows):
        if x == rr:
            b_vector[x] = old_b_vector[x]/old_matrix[rr][rc]
        else:
            b_vector[x] = old_b_vector[x] - ((old_b_vector[rr] *
old_matrix[x][rc]) / old_matrix[rr][rc])

    # Метод прямоугольников (Ответ)
    z_answ = old_z_answ - ((old_z_vector[rc] * old_b_vector[rr]) /
old_matrix[rr][rc])
    return matrix, b_vector, z_vector, basis, z_answ,

def solution(b_vector, z_vector, basis, z_answ, target):

```

```

sol = "Z("
for i in range(len(z_vector) - len(b_vector)):
    if i in basis:
        sol += str(b_vector[basis.index(i)])
    else:
        sol += "0"
    if i < len(z_vector) - len(b_vector) - 1:
        sol += ", "
sol += ")"
sol += " = "
sol += str(z_answ*target)
return sol

```

```

def com_solution(solutions, final_basis, z_vector, z_answ, target):
    num_vars = len(z_vector)
    num_basis = len(final_basis)
    num_real_vars = num_vars - num_basis

    # Все уникальные решения
    all_solutions = []

    for b_vec, basis, _ in solutions:
        sol_vec = [Fract(0) for _ in range(num_real_vars)]
        for i, var in enumerate(basis):
            if var < num_real_vars:
                sol_vec[var] = b_vec[i]

        if all(v >= 0 for v in sol_vec):
            if sol_vec not in all_solutions:
                all_solutions.append(sol_vec)

    # Фильтр нулевого решения
    non_zero_solutions = []
    for sol in all_solutions:
        not_null = False
        for v in sol:
            if v != 0:
                not_null = True
                break
        if not_null:
            non_zero_solutions.append(sol)

    if len(non_zero_solutions) >= 2:
        all_solutions = non_zero_solutions

    if len(all_solutions) == 1:
        sol = all_solutions[0]
        strs = []
        for val in sol:
            strs.append(str(val))
        all_x = ", ".join(strs)
        answer = "Z(" + all_x + ") = " + str(z_answ * target)
        return answer

```



```

# Сортировка решений по x1
all_solutions.sort(key=lambda x: float(x[0]))

sol_min = all_solutions[0] # решение с минимальным x1
sol_max = all_solutions[-1] # решение с максимальным x1

# Разица между макс. и мин. для каждого x
diff = []
for i in range(num_real_vars):
    d = sol_max[i] - sol_min[i]
    diff.append(d)

# Формирование ответа
result = "Z("
for j in range(num_real_vars):
    if diff[j] == 0:
        # Переменная не меняется
        result += str(sol_min[j])
    else:
        # Переменная меняется
        if diff[j] > 0:
            if diff[j] == 1:
                result += f"{sol_min[j]} + λ"
            else:
                result += f"{sol_min[j]} + {diff[j]}·λ"
        else:
            abs_diff = abs(diff[j])
            if abs_diff == 1:
                result += f"{sol_min[j]} - λ"
            else:
                result += f"{sol_min[j]} - {abs_diff}·λ"

    if j < num_real_vars - 1:
        result += ", "

result += f") = {z_answ * target}"
result += f"\nгде λ ∈ [0, 1] - параметр"

return result

```

```

def isInf(z_vector, basis):
    for i in range(len(z_vector)):
        if (z_vector[i] == 0) and (i not in basis):
            return True
    return False

```

```

=====
=====

```

```

def AmbivalentSimplex(matrix, b_vector, z_vector, target):

```

```

    NO_SOLUTION = -1
    ONE_SOLUTION = 0

```

```

INF_SOLUTION = 1
status = ONE_SOLUTION

print()
print("Исходная задача:")
print_problem(matrix, b_vector, z_vector, target, False)
print()
# ШАГ 1:
# Домножаем строки матрицы на -1 при необходимости.
# Формируем базис
step = 0
basis = []
z_answ = 0
for x in range(len(matrix)):
    for y in range(len(matrix[x])):
        if matrix[x][y] == -1 or matrix[x][y] == 1:
            if (isBasic(x, y, matrix, z_vector)):
                basis.append(y)
                if matrix[x][y] < 0:
                    mulRowVal(matrix[x], -1)
                    b_vector[x] *= -1
                break

print("Преобразованная задача:")
for i in range(len(z_vector)):
    z_vector[i] *= target
if target > 0:
    print_problem(matrix, b_vector, z_vector, target, False)
else:
    print_problem(matrix, b_vector, z_vector, target, True)
print()

# ШАГ 2:
# Формируем Z-строку в таблице
for i in range(len(z_vector)):
    z_vector[i] *= -1

x_sols = []
# ШАГ 3:
# Основной цикл поиска решения:
while(True):
    step+=1
    end = noNegative(b_vector)
    x_sols.append([b_vector, basis, z_answ])
    print()
    print("Таблица № " + str(step) + ":")
    if end:
        resolve_row = None
        resolve_col = None
        co_vector = None
    else:
        resolve_row = find_res_row(b_vector)
        co_vector = compute_co(matrix[resolve_row], basis, z_vector)
        resolve_col = find_res_col(co_vector)

```

```

        if resolve_col < 0: status = NO_SOLUTION
        if isInf(z_vector, basis): status = INF_SOLUTION
        printTable(matrix, basis, b_vector, z_vector, z_answ, co_vector,
resolve_row, resolve_col, target)
        print("Соответствующее решение:")
        print(solution(b_vector, z_vector, basis, z_answ, target))
        if (end or status == NO_SOLUTION): break
        print()
        print(f'Разрешающий элемент: {matrix[resolve_row][resolve_col]}')
        print(f'Выводим из базиса x{basis[resolve_row] + 1}')
        print(f'Вводим в базис x{resolve_col + 1}')
        matrix, b_vector, z_vector, basis, z_answ = new_table(matrix,
b_vector, z_vector, z_answ, resolve_row, resolve_col, basis)

# ШАГ 4: Вывод итогового решения
print()
print("Итоговое решение:")

if status == NO_SOLUTION:
    print("Нет решения!")
    return

if status == INF_SOLUTION:
    print("Решений бесконечно много:")

answer = "Z_"
if target > 0:
    answer += "max"
else:
    answer += "min"
answer += " = "

if status == INF_SOLUTION:
    answer += com_solution(x_sols, b_vector, z_vector, z_answ, target)
else:
    answer += solution(b_vector, z_vector, basis, z_answ, target)
print(answer)

#=====
=====

if __name__ == "__main__":
    path = ".\examples"
    print(80 * "-")
    print( (30 * " ") + "РГР" + (30 * " ") )
    print( (10 * " ") + "Решение ЗЛП двойственным симплекс-методом" + (20 *
" ") )
    print(80 * "-")
    while(1):
        print("\nВведите имя файла с матрицей (или exit, чтобы выйти): ")
        name = input()

        if (name == "exit"):
            print("Завершение работы...")
            break

```

```
if not name.__contains__(".txt"):
    name = name + ".txt"
f_matrix = open(f'{path}\\{name}', "r")
s_matrix = f_matrix.read()
f_matrix.close()

matrix, b_vector, z_vector, target = parse_matrix(s_matrix)
AmbivalentSimplex(matrix, b_vector, z_vector, target)
```